CSE 150: Introduction to Computer Networks

Chris Parsa

Chapter 2 Part 2

1

Current Assignments and Upcoming

- Chapter 2
  - Reading Assignment: Chapter 2.4 (DNS), 2.5 (Peer to peer, skip Bit Torrent), 2.7 (will need for project)
  - Chapter 2 practice problems are posted
- Lab3: Saturday at 11:59pm
- Group Assignment 3: Sunday at 5pm

2

Test 1

- Test 1 -- Tuesday October 22
  - Form study groups!!
  - Paper Exam, closed book, closed notes
  - Calculator, ID
  - Questions on Ed, office hours, tutoring
- Refer to Canvas Practice,Practice, Practice module for reading and problem assignments
- Material covered:
  - Textbook: Ch1 and Ch 2.1 and 2.2
    - No DNS, for HTTP as far as we discuss TODAY
  - Labs 1,2, 3
  - **Practice problems listed in Canvas Practice,Practice, Practice module (R & P) and "Homework" in lecture slides**
  - Class discussions / warmups / practice w neighbor / QR code practice in class
  - Group Assignments 1,2,3

3

Late Days

- Check # on Canvas
- 3 grace days, any reason
  - After that, -10 points per day

Regrade Policy

- Policy posted on Canvas – lab module (TAs)
- Group Assignments: see Chris

4

Chapter 2: outline

2.1 principles of network applications ✓
- app architectures ✓
- app requirements ✓
- Interface to Transport Layer ✓

2.2 Web and HTTP
- Basic HTTP ✓
- 2.2.4 Cookies
- 2.2.5 Web caching and Conditional GET

~~2.3 electronic mail~~
- ~~SMTP, POP3, IMAP~~

2.4 DNS – fundamental protocol

2.5 P2P applications

2.7 socket programming with UDP and TCP

5

More about HTTP

- HTTP is stateless
  - All HTTP requests are independent of each other
  - No history kept
  - Server doesn't need to manage sessions
  - Scalability

But, history is useful!

6

Lou Montulli (Netscape)

7

User-server state: Cookies

Most major Web sites use cookies to maintain some state between user transactions

(designed by Netscape's Lou Montulli)

Benefit of cookies:

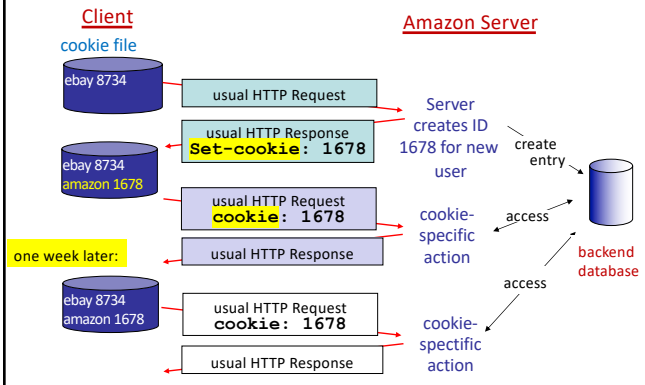
- Authorization
- Shopping carts
  - Recommendations
- User session state (Web e-mail)

An Example:

- A user accesses Internet always from the same PC
- Visits specific e-commerce site for first time
- How can the site "remember" her?
  - Set a Cookie

8

Example: Cookies: keeping "state"



9

HTTP Support for Cookies

Four components:

1) Server sets cookie with HTTP header set_cookie: x in its response message
2) When send requests, per domain, Clients return cookie using HTTP header: cookie: x
3) Client stores cookies in Cookie file, managed by user's browser
4) Web server keeps track of whatever it can in its Back-end database

Review: GET Request and Response with cookie and set-cookie header
Read book

10

Jumping to Ch 2.4

- DNS (because of Lab 4)
- Then we will go back to Ch 2.2

11

11

Complexity/computing at the Edge

- Apps run on Hosts/end systems on the Edge of the Network
- IP Routers in the Core
- By design!!
  - Keep Core simple
  - Network Complexity/computation pushed to Edge

file transfer
e-mail
Web documents / cookies
real-time audio/video

stored audio/video
interactive games
text messaging

12

12

Domain Name System (DNS)

- Lab next week

Mnemonic (aliases)

To recall the names of the first 20 elements in the periodic table:

Harry, he likes beer by cupfuls, not over frothy, never nasty mugs allowed. Since past six closing, are kegs cancelled?

(H, He, Li, Be, B, C, N, O, F, Ne, Na, Mg, Al, Si, P, S, Cl, Ar, K, Ca.)

13

Why DNS?

We use names!

People deal better with names .. not numbers…

- SSN, passport #, birth name

Internet hosts/servers, routers:

- Use 32 bit IP address to address datagrams

Think about how you reach web servers or send email

→IP Address or domain name?

14

Need Translation

Who remembers 411? Telephone book?

Q: How do we map between IP addresses and host name ?

- "Phonebook" for hostnames
  - Originally a file →host.txt
  - Scale as Internet grows
  - ~1983/4 DNS

15

Components of DNS

Why distributed?
Why hierarchical?

Distributed hierarchical database

- Replicated servers, distributed geographically
- Hierarchy of name servers (each has partial information about the name space
  - Store Resourse Records (RR)
  - www.lemonade.fr www.amazon.com

Application-layer Protocol

- Core Internet protocol
- Client-Server Architecture
  - Clients query Servers to "resolve" names, obtain RR
    - e.g. Name->IP address translation
- Defines messages
  - Syntax, format, semantics, response

An example of keeping complexity at network's "edge"

16

What is a DNS Nameserver?

Client-Server

A server that:

- Listens on port 53 for queries
- Stores DNS resource records (RR) for a domain
- Responds with answers (RRs) to queries against its database
  - Look at RRs in Lab4
  - Answers can be Final Answer or a Referral

17

DNS services

- Mapping:
  - Host name to IP address translation
    - www.whitehouse.gov
  - Host aliasing
    - Canonical and Alias (mnemonic) names
  - Mail server aliasing
    - jgebiden@whitehouse.gov
- Load distribution
  - Replicated content on distributed servers
  - Multiple IP addresses correspond to one name
    - DNS server can return a list of IPs
- All Internet-connected devices are running a DNS client AKA "resolver"
  - "local name server" specified by your access network (ISP) on your behalf. → starting point for Queries

18

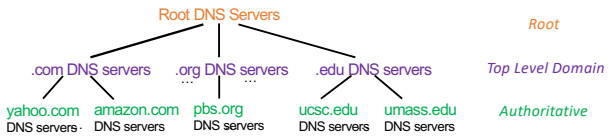
DNS Distributed Hierarchical Database

- Hierarchy:
  - Minimum 3 levels
    - 1
    - 2
    - 3
    - Can further partition
  - Each layer has specific knowledge
- Replicate and Distribute

19

DNS: Hierarchical database

How to "resolve" www.amazon.com ?



- Client queries **root** →gets IP Address for **.com** DNS server
- Client queries **.com** → gets IP Address for **amazon.com** TLD DNS server
- Client queries **amazon.com** Authoritative DNS server →gets IP address for www.amazon.com
- Benefit: further partition!

20

Think it over…

Servers are replicated and distributed geographically

Q: Why not centralize DNS?

- Distant centralized database
  - Who gets to be close?
- Single point of failure
  - Traffic volume
  - Maintenance

Q: Why hierarchy?

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries per day!!

22

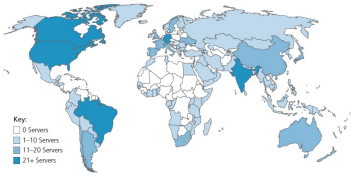
DNS: root name servers

Top of hierarchical distributed database

- Official, contact-of-last-resort
  - Can always start query here if needed
- Holds address of every TLD server
  - Root returns IP Address of either:
    - TLD or
    - Authoritative name server
    - or whatever nameserver is cached

13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)

ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain



23

TLD: Top Level Domain Servers

organizations, groups, counties

- Responsible for suffixes in name
  - .com, .org, .net, .edu, .gov, .jobs, .museums
  - all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Responsible for Authoritative servers within a suffix
  - Returns their IP Address!
  - e.g., know IP for amazon.com Authoritative server
- Network Solutions: authoritative registry for .com, .net
- Educause: .edu TLD

25

Authoritative servers

"I'm responsible"

Important: An organization's own DNS server(s), responsible for all mappings of publicly addressable hosts within its domain

e.g., Hostname → IP mapping

- Can be maintained by organization itself or service provider
- Publicly reachable named hosts must have a Resource Record

Note: Cached mappings are NOT considered to be "Authoritative"

Benefit:

- Updates/changes done locally
- Can further sub-divide the name and install additional name servers *

26

Authoritative servers

"I'm responsible"

27

Local DNS name servers

First contact

- AKA "default name server"
  - Does not strictly belong to hierarchy
  - Each ISP (residential ISP, company, university) has one
- When host makes DNS query, query is first sent to local DNS server
  - Has local cache of recent RRs (but may be out of date!)
  - Acts as proxy, forwards client queries into hierarchy
    - gethostbyname()

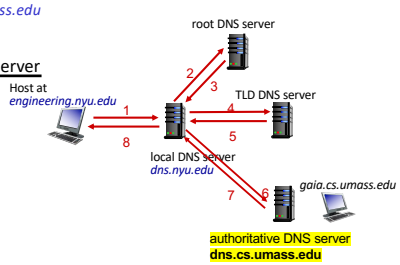
28

DNS name resolution: iterated query

Example: Host at *engineering.nyu.edu* wants IP address for *gaia.cs.umass.edu*

Begin with query to Local Nameserver

Iterated query:
- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"
- Bulk of work done by local name server



Time to resolve DNS?
These queries use UDP

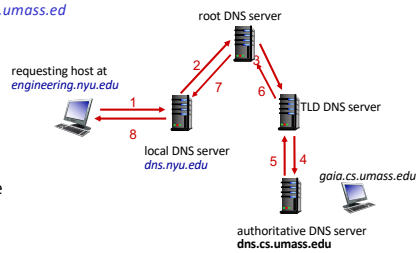
4 queries
- 3 iterative
- 1 recursive

29

DNS name resolution: recursive query

Example: Host at *engineering.nyu.edu* wants IP address for *gaia.cs.umass.ed*

Recursive query:
- Puts burden of name resolution on contacted name server
- Query is resolved before response given
- Heavy load at upper levels of hierarchy
  - Might not be allowed



30

DNS Caching

- Caching ubiquitous
- Once (any) name server learns a mapping, → *cache it!*
  - Local Nameservers typically cache TLD servers
  - Root rarely contacted
- BUT, **only** Authoritative NS guaranteed to have fresh/true mapping
- Note: Cached mappings are NOT considered to be "Authoritative"
- Cached entries may be *out-of-date*
  - If named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - timeout (disappear) after some time → why?

31

Why Cache?

- Benefits
- Drawbacks
- Talk to your neighbor!

32

Using DNS

gethostbyname()

35

Using DNS

37

DNS records

DNS: distributed database stores Resource Records (RR)

Use DNS Query to obtain RR:

RR format: (`name, value, type, ttl`)

type=A
- `name` = hostname
- `value` = IP address

type=NS
- `name` = domain (e.g., foo.com)
- `value` = hostname of authoritative name server for the domain

type=MX
- `value` = name of mailserver associated with `name`

type=CNAME
- `name` = alias name (easy to remember)
- `value` = canonical (real) name

(www.ibm.com, servereast.backup2.ibm.com, CNAME, TTL)

- e.g., www.ibm.com is really servereast.backup2.ibm.com

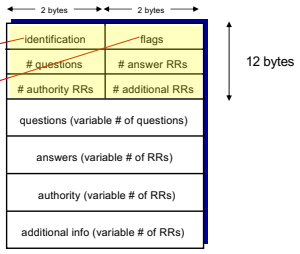
What is needed now?

39

DNS protocol messages

DNS *Query* and *Reply* messages, both have same *format:*

message header:
- identification: 16 bit # for query/ reply matching
- flags:
  - query or reply
  - recursion desired
  - recursion available
  - reply is Authoritative



41

DNS protocol messages

RR format: (name, value, type, TTL)

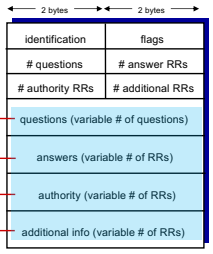
DNS *query* and *reply* messages, both have same *format:*

Name, type fields for a query

RRs in response to query (value)

records for authoritative servers

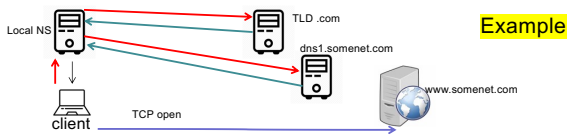
additional " helpful" info that may be used



42

Back to Loading Web Page

43



Client wants to load a webpage from www.somenet.com

- 1. Resolve DNS to get server's IP
- 2. Open TCP connection
- 3. Send HTTP GET message

DNS Resolution:

- Client sends **recursive** query to Local NS
- Local NS: sends **iterative** query to TLD (what was skipped?)
- TLD: returns RRs to Local NS:
  - NS: somenet.com dns1.somenet.com (authoritative server for somenet.com)
  - A: dns1.somenet.com 212.212.212.1
- Local NS: sends query to dns1.somenet.com @ 212.212.212.1 and in reply receives A record: www.somenet.com 212.212.212.45
- Local NS: returns IP address to web client
- This took some time!! What does web client do now?

44

How much time to load web page?

- What are the steps?
  - Resolve DNS
  - Open TCP Connection to web server
  - Request Base HTML page
  - Request Objects
    - Persistent or non-persistent TCP connection
    - parallel
- *e.g., Ch2* sample problem #3

45

Chapter 2: outline

2.1 principles of network applications ✓
- app architectures ✓
- app requirements ✓
- Interface to Transport Layer ✓

2.2 Web and HTTP
- Basic HTTP ✓
- 2.2.4 Cookies
- 2.2.5 Web caching

~~2.3 electronic mail~~
- ~~SMTP, POP3, IMAP~~

2.4 DNS – fundamental protocol ✓

2.5 P2P applications

2.7 socket programming with UDP and TCP

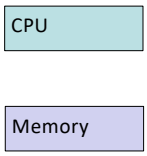
46

Cache

- What is a proxy?
- What is a cache?
- Some caches in computer networks:
  - DNS and web cache

47

Computer Architecture → Memory Cache

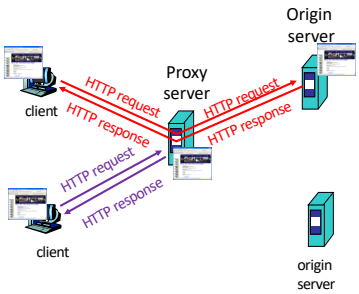


49

Web caches (proxy server)

Goal: Satisfy client request without involving origin server

- User configures browser: All HTTP requests sent to cache
- What is in the cache?
  - Content requested previously by other users



What is the difference between replication and caching?

51

Web caches

Initial Goals: Satisfy client requests on behalf of the origin server

→ speed things up

→ relieve load on origin server

→ and more (in example below)

53

Web cache – 2 hats!

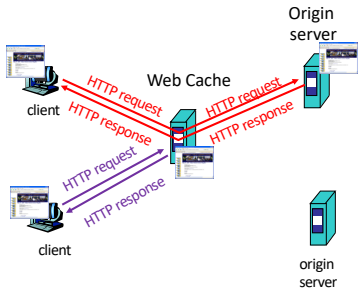
Review **Conditional Get**

Cache wears different hats!

Web cache acts as both client and server

- Server requesting client
- Client to origin server

- How does Cache get loaded?



Remember Conditional GET

54

Considerations

- How is cache loaded?
- Is the cache well utilized?
- Hit Rate: fraction of content requests stored in the cache
  - Effectiveness of cache can depend on
    - Cache size vs. cache management strategies
      - Replacement strategies when full
      - Pre-emptive fetching
    - LoR (Locality of Reference)
      - Who Is Using the Cache?
    - Static or dynamic content ?
- What type of data is good for caching?
  - Static? Dynamic?
- Is content up to date? → all caches
- Cache placement

57

Where should cache be placed?

59

59

Where should cache be placed?

- "Close" to client
- "In the middle"
- "Close" to server

- Savings on performance will depend on placement

60

60

Cache Close to Client

Typically, cache is installed by ISP (university, company, residential ISP) inside network

Why Web caching here?

- Reduce response time for client requests
  - cache is closer to client in high speed network
- Reduce traffic on an institution's access link

61

Cache inside Access Network

close to client

62

Cache In the Middle (Access ISP)

64

Cache close to Origin Server

66

Example: Let's see how to get:

- Faster response time for client
- Reduce traffic on local network's access link

68

Reduce Client's Delay to load web content on Origin Server

Components of delay:

Internet delay + access delay + LAN delay

Internet Delay: RTT (ISP router → origin server)

= ~2sec

Access Delay:

- If link not over-utilized (load/intensity <0.8), delay over access link = ~10 msec
- If over-utilized (intensity>=0.8), delay over access link delay increases to many seconds * (possibly pkts dropped)

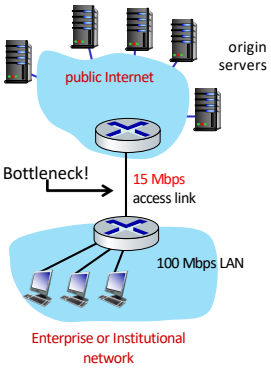
LAN Delay: high speed, low delay

- Host to Gateway
- ~10msec

Total Delay under heavy load

Delay >> ~ 2 sec + sec* + 10msec,

i.e. many seconds to even minutes



69

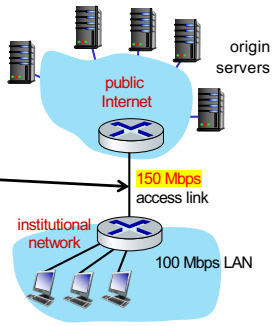
Example (cont.)

1. Possible improvement

Often a costly upgrade:

Pay access ISP to upgrade access link rate: (15 → 150 Mbps)

→reduce intensity/load on link



Total Average Delay

= Internet delay + access delay * + LAN delay

~ 2 sec + 10msecs + 10msecs

~ 2.02 seconds

*low intensity on access link brings delay down to ~10msec.

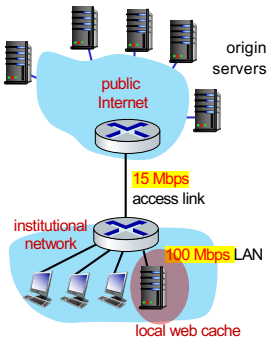
70

Example (cont.)

2. Another possible solution: Install cache

Hit Rate

- Fraction of requested content stored by the cache

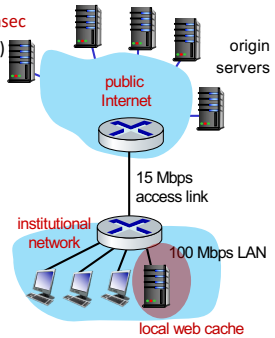


71

Caching example (cont.)

Suppose cache's Hit Rate = 0.4

- Access link not overutilized, delay = ~ 10msec (total delay =~ 2 sec + 10msecs + 10msecs)
- (Notice: LAN and Access link delays negligible compared to Internet delay)



Result:

- 40% requests satisfied at cache (**hit**),
- 60% requests satisfied at origin (**miss**)

Total Average Delay

= miss hit
= .6*(2.02 sec) + .4*(0.01 sec)
= ~ 1.2 secs → big improvement!

- Cost: cheap!

Decrease user delay, Internet traffic and server load, no costly upgrade

72

Further Considerations

Placement

- Is geographically close always best?
- CDNs
  - Internet dense with caches: enables "poor" content providers to effectively deliver content based on demand

74